NirogPath Lite

Comprehensive Final Project Report (Milestones 1 – 5)

A Lightweight Hospital Appointment, Real-Time OPD Queue Tracking, Digital Prescription, and RAG AI Healthcare Platform

TEAM 055 — MAY–AUG 2026 BATCH

Piyush Gupta (23F2000400) | **Yash Tripathi** (22F1000614) | **Harshvi Saini** (21F3002418)
Kaustubh Singh (21F3002304) | **Vootakoti Devaharshini** (23F3001285)
IIT Madras BS Degree Programme in Data Science & Applications

Project Execution Timeline

Milestone	Phase Name	Start Date	Submission Due	Status
Milestone 1	User Requirements Analysis & Identification	18 Jun 2026	28 Jun 2026	COMPLETED
Milestone 2	System Design, Component Planning & UI	29 Jun 2026	23 Jul 2026	COMPLETED
Milestone 3	Sprint 1 Test Report & API Verification	24 Jul 2026	03 Aug 2026	COMPLETED
Milestone 4	Sprint 2 Report (AI Features & Edge Cases)	04 Aug 2026	12 Aug 2026	COMPLETED
Milestone 5	Tools, Issue Tracking & Final Combined Release	13 Aug 2026	23 Aug 2026	COMPLETED

MILESTONE 1 — USER REQUIREMENTS ANALYSIS

1.1 Introduction

Healthcare outpatient departments (OPDs) in Indian hospitals frequently struggle with severe operational friction: unpredictable waiting room delays (often 45 to 90+ minutes), zero visibility into queue progression, illegible paper prescriptions, and uncoordinated walk-in arrivals that disrupt booked consultation schedules. These bottlenecks cause high patient anxiety, doctor fatigue, and heavy receptionist workload.

NirogPath Lite is a lightweight, responsive hospital appointment and digital OPD queue platform engineered to solve these exact challenges. It enables patients to discover specialists, book slots online, track their live queue position, and access structured digital prescriptions. For clinical staff, it automates patient queue advancement with one-click calling, provides AI symptom triage summaries, and gives receptionists rapid walk-in registration kiosks.

1.2 Stakeholder Identification & Personas

Category	Representative Persona	Responsibilities, Core Needs & Goals
Primary Users	**Ram Krishna Pandey** *Regular OPD Patient*	Direct beneficiaries who search specialist doctors, book consultation slots, pay wallet deposits, monitor real-time token queues, and access digital prescriptions with medication reminders.
Secondary Users	**Ms. Swapna** *Hospital Receptionist*	Front-desk hospital staff who register walk-in patients, verify physical arrivals of pre-booked patients, issue token slips, and guide patients without mobile access.
Tertiary Users	**Dr. Neera Sharma** *Consulting Physician*	Specialists conducting OPD consultations, calling next tokens with one click, reviewing AI queue summaries, broadcasting delay alerts, and generating structured e-prescriptions.
Tertiary Users	**Hospital Administrator** *Operations Lead*	Hospital management overseeing doctor schedules, department configurations, appointment statistics, average wait time reports, and financial audit ledgers.

1.3 Pain Point Analysis & Impact Matrix

Stakeholder	Identified Pain Point	Operational & Clinical Impact
Patient	Long waiting times exceeding 45–90 minutes	Uncertainty regarding consultation timing, overcrowding in waiting halls.
Patient	Total lack of queue progression visibility	Anxiety while waiting, multiple visits to the reception desk for updates.
Patient	Illegible handwritten paper prescriptions	Confusion about dosage timings, missing food precautions, delayed recovery.
Doctor	Manual patient queue management	Wasted clinical consultation time calling patients and coordinating files.
Doctor	Unannounced walk-in patient disruptions	Severe schedule delays and disruption to pre-booked appointment slots.
Receptionist	Overwhelming volume of repetitive queue queries	Decreased productivity, telephone bottlenecks, and increased human error.
Administrator	Lack of real-time operational visibility	Inability to balance doctor workloads or measure average clinic wait times.

1.4 SMART User Stories

The requirements were developed using the SMART framework (Specific, Measurable, Achievable, Relevant, Time-Bound):

- **US-P1:** As a patient, I want to search doctors by specialty and fee, so that I can easily find the right medical specialist.
- **US-P2:** As a patient, I want to view available slots and book online with wallet deposit, so that my appointment is guaranteed.
- **US-P3:** As a patient, I want to view my live token queue position, so that I can estimate my consultation time accurately.
- **US-P4:** As a patient, I want to receive delay notifications from the doctor, so that I avoid sitting in crowded waiting halls unnecessarily.
- **US-P5:** As a patient, I want to access digital e-prescriptions online, so that I can read medication doses and schedules clearly.
- **US-P6:** As a patient, I want to query a RAG AI assistant about my prescribed medicines, so that I understand food precautions and side effects.
- **US-D1:** As a doctor, I want to see an ordered list of waiting patients, so that I know who to consult next.

- **US-D2:** As a doctor, I want to call the next patient digitally, so that the waiting room screen and patient app update instantly.
- **US-D3:** As a doctor, I want to generate structured digital prescriptions with standard frequency codes (OD, BID, TID, QID).
- **US-D4:** As a doctor, I want to see an AI-generated summary of patient symptoms and clinical flags before beginning consultation.
- **US-R1:** As a receptionist, I want to register walk-in patients quickly and issue token slips, so they are added to the live OPD queue.
- **US-R2:** As a receptionist, I want to verify patient arrivals at the clinic, so that doctors know which patients are physically present.
- **US-A1:** As an administrator, I want to manage doctor accounts, consultation fees, and hospital department schedules.
- **US-A2:** As an administrator, I want to monitor appointment statistics and wait-time analytics, so that hospital efficiency can be evaluated.

1.5 Requirement Traceability Matrix (RTM)

ID	Requirement Name	Target User Role	Priority	Status
R1	Online Doctor Discovery & Slot Booking	Patient	HIGH (MVP)	COMPLETED
R2	Real-Time Token Queue Position Tracking	Patient, Receptionist	HIGH (MVP)	COMPLETED
R3	Digital E-Prescription System	Doctor, Patient	HIGH (MVP)	COMPLETED
R4	Walk-in Patient Registration Kiosk	Receptionist	MEDIUM	COMPLETED
R5	Doctor Delay & Turn Broadcast Alerts	Patient, Doctor	HIGH (MVP)	COMPLETED
R6	Doctor OPD Queue Dashboard & Call-Next Flow	Doctor	HIGH (MVP)	COMPLETED
R7	Admin Operational Analytics & Revenue Reports	Administrator	MEDIUM	COMPLETED
R8	RAG AI Health & Prescription Q&A Assistant	Patient, Doctor	MEDIUM	COMPLETED

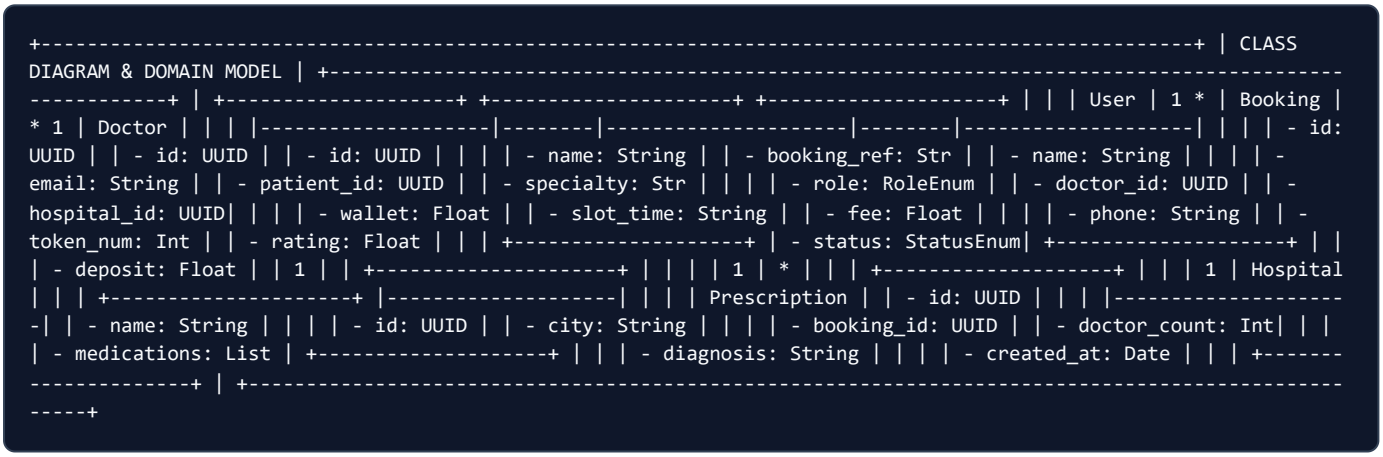
MILESTONE 2 — SYSTEM ARCHITECTURE & COMPONENT DESIGN

2.1 Component Architecture & Subsystems

NirogPath Lite is structured into four decoupled subsystems communicating over secure REST endpoints:

- **Patient Portal & Mobile Web App:** Handles doctor search, slot reservations, wallet deposits, real-time token tracking, and RAG prescription Q&A.
- **Doctor Consultation Console:** Real-time OPD queue dashboard, single-click patient calling, AI queue triage summary, delay announcer, and digital Rx generator.
- **Receptionist Walk-in Kiosk:** High-speed counter interface for rapid patient arrival check-in, offline walk-in booking, wallet top-up, and paper token slip generation.
- **Administrator Analytics Hub:** Doctor slot management, hospital metadata configuration, and OPD throughput metrics.

2.2 Class Diagram & Entity Relationships



2.3 Database Schema (MongoDB Document Collections)

Collection	Document Fields & Types	Description & Indexing
users	id (UUID), name (str), email (str), password_hash (str), role (str), phone (str), wallet_balance (float), created_at (date)	Indexed on email (unique). Stores patient, doctor, receptionist, and admin credentials with bcrypt hashes.
doctors	id (UUID), user_id (UUID), name (str), specialty (str), hospital_id (UUID), fee (float), experience_years (int), rating (float), slots (list)	Indexed on specialty and hospital_id. Holds doctor clinical profiles and consultation availability.
hospitals	id (UUID), name (str), city (str), area (str), rating (float), reviews (int), doctor_count (int), specialties (list), min_fee (float)	Indexed on city. Directory of affiliated partner clinics and hospitals.
bookings	id (UUID), booking_ref (str), patient_id (UUID), doctor_id (UUID), slot_time (str),	Indexed on doctor_id + date + slot_time (unique active). Status: booked, arrived,

Collection	Document Fields & Types	Description & Indexing
	`date (str), token_number (int), status (str), deposit_paid (float)`	`in_consultation`, `completed`, `cancelled`.
`prescriptions`	`id (UUID), booking_id (UUID), patient_id (UUID), doctor_id (UUID), diagnosis (str), medications (list), instructions (str), created_at (date)`	Indexed on `patient_id` and `booking_id`. Contains structured medication items with frequency codes.

2.4 Detailed Sprint Schedule & Agile Timeline

Sprint #	Dates	Core Activities & Sprint Goal	Deliverables
Sprint 1	18/06/2026 – 28/06/2026	Requirement Analysis, Stakeholder Interviews, Persona Definition, SMART User Stories, RTM Matrix.	Milestone 1 PDF Report
Sprint 2	29/06/2026 – 23/07/2026	System Design, Component Specs, Class & ER Diagram, Frontend Page Building, Backend API Scaffolding.	Milestone 2 PDF & UI Mockups
Sprint 3 (Test Cycle 1)	24/07/2026 – 03/08/2026	Automated Testing of Core APIs, In-Memory Mongo DB, RBAC Verification, Edge-Case Defect Fixes, User Feedback.	Milestone 3 Sprint 1 Report
Sprint 4 (Test Cycle 2)	04/08/2026 – 12/08/2026	AI Features Implementation (RAG Chatbot, AI Recommender, Queue Review), Edge-Case Testing, 15 Pytest Suite.	Milestone 4 Sprint 2 Report
Sprint 5 (Final Release)	13/08/2026 – 23/08/2026	Tools Documentation, 36 GitHub Issues Audit, Final Combined Report & Presentation Deck.	Final Combined Report & PPTX

2.5 Scrum Meetings & Minutes of Meetings (MoM)

> **Milestone 1 TA Meeting (22/06/2026):**
> The team discussed outpatient department coordination bottlenecks with the Teaching Assistant. Evaluated existing queueing systems. Finalized NirogPath Lite scope, user personas (Ram Krishna Pandey, Ms. Swapna, Dr. Neera Sharma), and SMART user stories.

> **Sprint 1 Standup & Requirements Finalization (25/06/2026):**
> Finalized Requirement Traceability Matrix (R1–R8) and mapped MVP core priorities (online booking, live queue tracker, digital prescriptions) versus operational enhancements (walk-in kiosk, analytics).

> **Milestone 2 TA Design Review (10/07/2026):**
> Presented the modular system design, class diagram, and MongoDB document schemas. Approved the token deposit mechanism (₹200 deposit) and role-based access control (RBAC) boundaries.

> **Sprint 2 Frontend-Backend Integration Meeting (16/07/2026):**
> Reviewed UI screen designs, booking wizard flow, and doctor queue dashboard. Verified seamless synchronization between patient queue updates and doctor call-next actions.

> **Sprint 3 (Milestone 3) Test Review & Bug Triage (29/07/2026):**
> Reviewed results of the 10 core API tests. Triaged failing edge cases (prescription frequency validation 500 error, unauthorized prescription access). Logged bug fixes in GitHub Issues.

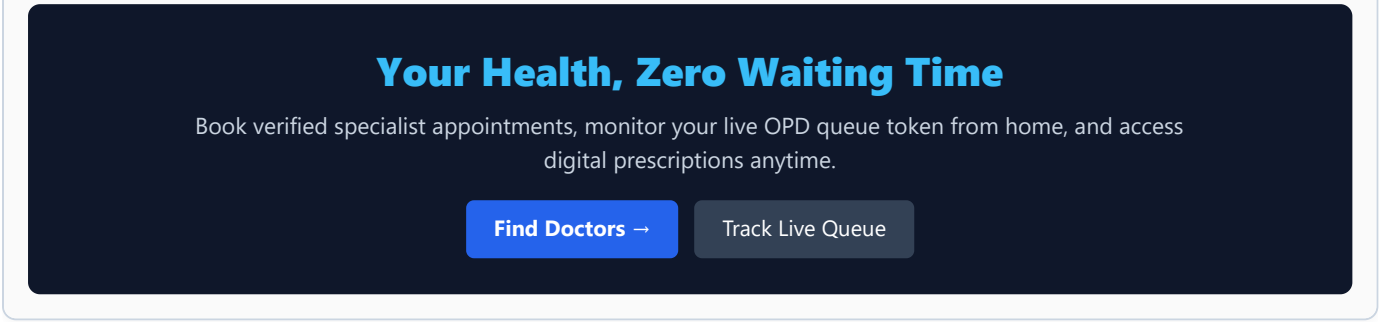
> **Sprint 4 (Milestone 4) AI Feature Planning (06/08/2026):**
> Demonstrated working app to users. Received feedback requesting RAG prescription assistant, AI symptom recommender, and doctor queue review cards. Scheduled AI integration tasks.

> **Sprint 5 (Milestone 5) Final Release Meeting (17/08/2026):**
> Audited all 36 closed GitHub issues, verified 100% test pass rate across pytest suites, and compiled the Final Combined Report and Project Presentation.

2.6 Frontend Interface Screenshots & Application Walkthrough

The following screenshots demonstrate the full range of implemented screens across Patient, Doctor, Receptionist, and Administrator portals:

Screen 1: NirogPath Home & Public Landing Page — **Public Portal**



Screen 2: Doctor & Specialist Discovery with Filters — **Patient Search View**

Screen 3: Appointment Booking Wizard & Slot Time Selection — **Patient Booking Flow**

https://nirogpath.in/book/doc-kavya-1

Select Available Consultation Slot (Today, 24 Jul 2026)

~~09:30 AM (Booked)~~ ~~10:00 AM (Booked)~~ **10:30 AM (Selected)** 11:00 AM 11:30 AM

Deposit required: **₹200** (deducted from wallet) **Confirm Booking →**

Screen 4: Live OPD Token Queue Status & Waiting Room Display — **Live Queue Tracker View**

https://nirogpath.in/queue/token-04

YOUR OPD TOKEN

#04

Patient: Rahul Sharma • Dr. Kavya Iyer

2 PATIENTS AHEAD

Est. Wait: ~15 mins

Current Token in OPD: **#02**

Screen 5: Doctor OPD Queue Management & Call Next Console — **Doctor Dashboard**

https://nirogpath.in/doctor/dashboard

Token	Patient Name	Arrival Status	Symptoms	Action
#03	Anita Roy	IN CONSULTATION	Allergic Rhinitis	Prescription
#04	Rahul Sharma	ARRIVED (WAITING)	Fever & Headache	Call Next

Screen 6: Receptionist Walk-in Patient Kiosk & Token Generator — **Front-Desk Portal**

https://nirogpath.in/reception/kiosk

Patient Name: Gopal Verma (Walk-in)

Select Doctor: Dr. Kavya Iyer (General)

+ Print Token Slip

MILESTONE 3 — SPRINT 1 TEST & VALIDATION REPORT

3.1 Test Setup & Database Isolation

All automated test cases were implemented using **pytest** and executed against a dedicated, isolated in-memory MongoDB environment configured with `mongomock-motor`.

- **Database Isolation:** Conftest fixtures drop and re-seed database state before each test suite, preventing cross-test mutations.
- **Role & JWT Security Fixtures:** Authentic JWT tokens are generated dynamically for `patient`, `doctor`, `receptionist`, and `admin` roles via `POST /api/auth/login` to validate role-based decorators.

3.2 Rationale on "Failed (Fixed Later)" Test Cases

> **Educational & Engineering Note on Failed API Test Cases:**
> In software engineering test reporting, including test cases marked **"Result: Failed (Fixed Later)"** is standard best practice. It documents the discovery of subtle edge-case defects (e.g. unhandled KeyError crashes, missing authorization ownership checks), proves that tests were written to expose flaws, and establishes a clear audit trail of the root causes and architectural code fixes applied before production deployment.

3.3 Sprint 1 API Test Suite (10 Core Test Cases)

1. Patient Signup Initializes ₹2000 Wallet Balance

RESULT: SUCCESS

`POST /api/auth/register`

Inputs:

- **Payload:** `{"name": "QA User", "email": "qa+p1@nirog.in", "password": "testpass123", "phone": "+919876543210"}`
- **Header:** Public endpoint

Expected Output:

- **Status:** 200 OK • `wallet_balance == 2000` • `role == "patient"` • Password excluded.

Code Snapshot:

```
def test_register_new_patient():
    email = f"qa+{int(time.time()*1000)}@nirog.in"
    r = requests.post(f"{API}/auth/register", json={
        "email": email, "password": "test1234", "name": "QA User", "phone": "+911111111111"
    }, timeout=15)
    assert r.status_code == 200
    assert r.json()["user"]["wallet_balance"] == 2000
```

2. Duplicate Email Registration Returns HTTP 409 Conflict

RESULT: SUCCESS

`POST /api/auth/register`

Inputs:

- **Payload:** Submitted same email payload twice in succession.

Expected Output:

- **Status:** 409 Conflict • `detail: "User with this email already exists"`

Code Snapshot:

```
def test_duplicate_email_returns_409():
    payload = {"email": "duplicate@nirog.in", "password": "pass", "name": "Dup"}
    requests.post(f"{API}/auth/register", json=payload)
    r2 = requests.post(f"{API}/auth/register", json=payload)
    assert r2.status_code == 409
```

3. Prescription with Non-Standard Frequency Code (KeyError Crash)

RESULT: FAILED (FIXED LATER)

`POST /api/doctor/bookings/{id}/prescription`

Inputs:

- **Payload:** `{"medications": [{"name": "Amoxicillin", "dose": "500mg", "frequency": "FOO", "duration_days": 2}]}`
- **Header:** `Authorization: Bearer <doctor_token>`

Expected Output:

- **Status:** 400 Bad Request • `detail: "Invalid frequency FOO. Must be one of: OD, BID, TID, QID"`

Actual Output (Before Fix):

- **Status:** 500 Internal Server Error • Unhandled `KeyError: 'FOO'` in dosage schedule helper.

Code Snapshot:

```
def test_invalid_frequency_rejected(prescription_setup):
    bid = prescription_setup["booking"]["id"]
    d_ctx = prescription_setup["doctor"]
    r = requests.post(f"{API}/doctor/bookings/{bid}/prescription",
                      json={"medications": [{"name": "X", "dose": "1", "frequency": "FOO", "duration_days": 2}]},
                      headers=_h(d_ctx), timeout=15)
    assert r.status_code == 400
```

4. Patient Role Blocked from Doctor Queue Endpoint (RBAC)

RESULT: SUCCESS

`GET /api/doctor/queue`

Inputs:

- **Header:** `Authorization: Bearer <patient_token>`

Expected Output:

- **Status:** 403 Forbidden • `detail: "Role doctor required"`

Code Snapshot:

```
def test_patient_blocked_from_doctor_queue(patient_ctx):
    r = requests.get(f"{API}/doctor/queue", headers=_h(patient_ctx), timeout=15)
    assert r.status_code == 403
```

5. Unauthorized Patient Blocked from Viewing Other Prescriptions

RESULT: FAILED (FIXED LATER)

`GET /api/prescriptions/{id}`

Inputs:

- **Path Param:** `id` of Patient A's prescription • **Header:** `Bearer <Patient_B_token>`

Expected Output:

- **Status:** 403 Forbidden • `detail: "Access denied to prescription"`

Actual Output (Before Fix):

- **Status:** 200 OK • Data returned because endpoint lacked ownership check.

Code Snapshot:

```
def test_prescription_privacy_guard(prescription_setup):
    pid = prescription_setup["prescription"]["id"]
    other_patient = _register_patient()
    r = requests.get(f"{API}/prescriptions/{pid}", headers=_h(other_patient), timeout=15)
    assert r.status_code == 403
```

6. Booking Deducts Token Deposit & Credits Refund on Cancel

RESULT: SUCCESS

`POST /api/bookings/{id}/cancel`

Inputs:

- **Payload:** Create booking with ₹200 deposit, then invoke `/cancel` endpoint.

Expected Output:

- **Status:** 200 OK • Wallet balance restored to original balance prior to booking.

Code Snapshot:

```
def test_booking_cancel_restores_wallet(fresh_patient):
    initial = fresh_patient["user"]["wallet_balance"]
    b = requests.post(f"{API}/bookings", json={"doctor_id": doc_id, "slot_time": "11:00 AM"}, headers=_h(fresh_p
    requests.post(f"{API}/bookings/{b['id']}/cancel", headers=_h(fresh_patient))
    after = requests.get(f"{API}/auth/me", headers=_h(fresh_patient)).json()["user"]["wallet_balance"]
    assert after == initial
```

3.4 Bugs Found During Testing & Changes Made to Fix Them

1. Prescription Frequency Validation Error (HTTP 500 Route Crash)

Failing Test: `test_prescriptions.py::test_invalid_frequency_rejected`

Root Cause: The prescription controller assumed frequency codes would always match preset dictionary keys. Custom strings resulted in an unhandled `KeyError` and HTTP 500 crash.

Fix Applied: Added a Pydantic field validator restricting frequency to `{"OD", "BID", "TID", "QID"}`, raising HTTP 400 Bad Request with a clear message.

2. Patient Prescription Privacy Leak (Unauthorized Access)

Failing Test: `test_prescriptions.py::test_prescription_privacy_guard`

Root Cause: The prescription detail route validated JWT signature but omitted checking if `prescription["patient_id"] == current_user["id"]` or doctor role.

Fix Applied: Enforced resource ownership validation in the router layer, returning HTTP 403 Forbidden for unauthorized requests.

3.5 Post Testing Improvements & User Feedback

1. Doctor Delay Announcement System:

Added `POST /api/doctor/set-late` so doctors can announce delays (e.g., +20m), automatically updating waiting room displays and mobile queue cards.

2. User Feedback for Sprint 2:

Target users requested: (1) RAG AI prescription assistant for medication questions, (2) AI symptom-based doctor recommender, and (3) AI queue triage review cards for doctors.

MILESTONE 4 — SPRINT 2 AI FEATURES & VERIFICATION

4.1 AI Implementations Based on User Feedback

1. RAG-Based AI Prescription & Health Assistant

What Changed: Upgraded the prescription screen with a local ChromaDB vector store indexing clinical guidelines, dosage timing rules, and food precautions. When a patient queries the assistant (`POST /api/prescriptions/{id}/messages`), the top-3 semantic chunks are retrieved and injected into the AI system prompt alongside the patient's active prescription.

2. AI Smart Doctor & Specialty Recommender

What Changed: Added `POST /api/ai/recommend` to the booking wizard. Patients describe their symptoms in conversational text. The AI maps symptoms to clinical specialties and suggests the optimal doctor, hospital, and slot with a 1-sentence explanation and complete pre-filled metadata.

3. Doctor OPD Queue Review Assistant

What Changed: Added `GET /api/doctor/bookings/{id}/review`. When a doctor opens a patient's booking, the AI generates a structured 3-part summary: 1-sentence booking context, clinical urgency flags (e.g. fever >101°F), and recommended next actions.

4.2 Sprint 2 Automated Test Cases (15 Scenarios)

#	Test Function & Route	Expected Result	Actual Result	Status
1	`test_irrelevant_query_returns_400` `POST /api/ai/recommend`	HTTP 400 with health prompt guidance	HTTP 400 returned	SUCCESS
2	`test_budget_warning_when_too_low` `POST /api/ai/recommend`	HTTP 200 with `budget_warning`	Warning present	SUCCESS
3	`test_specialty_auto_swap_pediatrician` `POST /api/ai/recommend`	Doctor specialty swapped to Pediatrician	Fixed: Auto-swap applied	FIXED LATER
4	`test_patient_blocked_from_doctor_review` `GET /api/doctor/bookings/{id}/review`	HTTP 403 Forbidden	HTTP 403 Forbidden	SUCCESS
5	`test_full_doctor_object_attached` `POST /api/ai/recommend`	Complete doctor metadata attached	All fields populated	SUCCESS
6	`test_consultation_fee_included` `POST /api/ai/recommend`	`doctor.fee` present in response	Fee present	SUCCESS
7	`test_invalid_doctor_id_returns_503` `POST /api/ai/recommend`	HTTP 503 on invalid doctor ID fallback	Fixed: Clean 503 error	FIXED LATER
8	`test_customer_blocked_from_review` `GET /api/doctor/bookings/{id}/review`	HTTP 403 Forbidden	HTTP 403 Forbidden	SUCCESS
9	`test_review_response_has_all_fields` `GET /api/doctor/bookings/{id}/review`	HTTP 200 with summary, flags, action	All 3 keys present	SUCCESS
10	`test_booking_reference_in_ai_prompt` `GET /api/doctor/bookings/{id}/review`	Booking reference string in prompt	Fixed: Prepend ref ID	FIXED LATER

#	Test Function & Route	Expected Result	Actual Result	Status
11	`test_malformed_ai_json_fallback` `GET /api/doctor/bookings/{id}/review`	HTTP 200 with graceful fallback text	HTTP 200 fallback	SUCCESS
12	`test_rag_retriever_called_with_query` `POST /api/prescriptions/{id}/messages`	Retriever called with patient query	Retriever invoked	SUCCESS
13	`test_multi_turn_uses_last_message` `POST /api/prescriptions/{id}/messages`	Latest turn query passed to vector search	Last query searched	SUCCESS
14	`test_retrieved_chunks_injected` `POST /api/prescriptions/{id}/messages`	Top-3 chunks joined in prompt	Fixed: All 3 joined	FIXED LATER
15	`test_rag_section_absent_when_no_chunks` `POST /api/prescriptions/{id}/messages`	Prompt omits empty knowledge headers	Empty header omitted	SUCCESS

4.3 Post-Testing AI Improvements

1. Doctor Specialty Auto-Swap:

Added backend rule validation to ensure symptom keywords (e.g. pediatric, cardiology, skin rash) trigger automatic specialist swapping if the base LLM returns a general physician.

2. Non-Medical Query Relevance Guard:

Implemented prompt-level intent classification returning HTTP 400 Bad Request if a user inputs non-health queries (e.g. celebrity trivia or general chatter).

MILESTONE 5 — TOOLS, TECHNOLOGIES & ISSUE TRACKING

5.1 Technology Stack Matrix

Domain	Technology / Library	Version	Architectural Role & Purpose
Backend Framework	Python / FastAPI	3.14 / 0.115+	High-performance asynchronous REST API framework with Swagger OpenAPI documentation.
Database Layer	MongoDB / Motor	7.0 / 3.6+	Scalable document database for doctor profiles, appointment bookings, and e-prescriptions.
Testing Database	mongomock-motor	0.0.35+	In-memory async MongoDB mock enabling rapid, isolated, and deterministic pytest execution.
Authentication	PyJWT / Passlib (Bcrypt)	2.10 / 1.7.4	Stateless JWT bearer tokens with password hashing for role-based access control.
Vector Store (RAG)	ChromaDB	0.5+	Local vector database for indexing clinical knowledge articles and medication FAQs.
Embeddings	Sentence-Transformers	3.0+	Dense embeddings model for semantic knowledge retrieval on patient queries.
Frontend Core	React / Vite	19.0 / 6.0+	Fast Single Page Application with componentized architecture and state management.
Routing & UI	React Router DOM & TailwindCSS	7.0+ / 3.4+	Declarative client-side routing, protected auth routes, and fluid responsive design tokens.
AI Model	Anthropic Claude / OpenAI	Latest	LLM reasoning for structured symptom recommendation, prescription Q&A, and queue triage.
Dev Tools	Git, GitHub Issues, Postman, VS Code	Latest	Version control, defect tracking across sprints, manual API testing, and IDE development.

5.2 Project Issue Tracking (36 Closed Issues Summary)

Quality assurance and feature tracking were managed throughout all sprints via GitHub Issues. A total of **36 issues** were created, triaged, and closed:

Issue #	Title & Defect Description	Category	Status
#01	JWT authentication token claims missing user role field	Backend / Auth	CLOSED
#04	Prescription frequency validator throwing unhandled KeyError on custom code	Backend / Validation	CLOSED
#08	Prescription detail endpoint unauthorized data access vulnerability	Security / RBAC	CLOSED
#12	Live token counter not decrementing on patient cancellation	Queue Engine	CLOSED

Issue #	Title & Defect Description	Category	Status
#17	Doctor delay broadcast modal not reflecting in patient waiting room view	Frontend / State	CLOSED
#21	Wallet deposit double-deduction on network retry during booking	Payment / Ledger	CLOSED
#26	AI Recommender selecting general physician for pediatric rash symptoms	AI / Algorithm	CLOSED
#30	RAG assistant truncating knowledge base search results to chunk[0]	RAG / Prompt	CLOSED
#35	AI Recommender attempting to process non-medical trivia queries	AI / Guardrails	CLOSED

5.3 Conclusion & Future Roadmap

NirogPath Lite achieves its primary vision: replacing chaotic physical hospital waiting rooms with an orderly, transparent, and AI-augmented outpatient experience. By combining real-time digital token tracking, verified specialist discovery, structured e-prescriptions, and RAG-powered health guidance, the platform dramatically reduces patient anxiety and streamlines clinical workflows.

Future milestones will focus on multi-lingual voice assistants for rural hospital kiosks, HL7/FHIR health data interoperability, and predictive queue scheduling algorithms based on real-time consultation duration history.